

M2Lean: Certificate-Based Interoperability Between Macaulay2 and Lean 4

Andrew Tawfeek*

August 2026

Abstract

M2Lean is a prototype interoperability layer between the computer algebra system *Macaulay2* and the *Lean* 4 proof assistant. *Macaulay2* exports computations together with checkable evidence—for example, ideal-membership cofactors, change-of-basis matrices, and standard representations of S-polynomials—in a versioned JSON protocol for exact polynomial algebra over $\mathbb{Q}$ and prime fields. A *Lean* library supplies executable checkers for all seven claim kinds and kernel-checked soundness theorems for polynomial identity, ideal membership, span inclusion, chain-complex composition, and GRevLex Gröbner and non-membership claims. Lex Gröbner and non-membership claims and the graded-complex claim remain executable-only.

We distinguish two outcomes that are often conflated. The `m2lean-check` executable produces a runtime verification report; its `proved` label means that the relevant checker has a formal soundness theorem, not that the particular report is a proof object. For the checked-in theorem examples, generated certificate literals are embedded in a theorem file, re-evaluated by *Lean*’s kernel, and passed to the soundness theorem. The prototype by `macaulay2` tactic instead obtains a live certificate during elaboration and constructs the corresponding soundness-theorem application. Both theorem paths yield a kernel-checked theorem.

The development includes a bridge-specific formalization of Buchberger’s standard-representation criterion, a degree-reverse-lexicographic monomial order and its agreement with the executable representation, and soundness for Gröbner-basis and non-membership certificates under that order. Case studies cover a scheme-theoretically empty intersection involving the twisted cubic, the 2026 Alpöge–Fable counterexample to the Jacobian conjecture, and Nullstellensatz certificates for graph non-colourability. In the runtime-path test suite, 15 valid documents are accepted and 20 malformed or corrupted documents are rejected. The evaluation is intended as a reproducibility and failure-detection study of the current prototype, not as evidence of research-scale performance.

1 Introduction

Computer algebra systems and proof assistants optimize for different tasks. A computer algebra system such as *Macaulay2* [GS24] provides mature algorithms for Gröbner bases, resolutions, and related computations. A proof assistant such as *Lean* 4 [dMU21], together with `mathlib` [The20], provides a small proof kernel and a language in which computational facts can be composed with abstract mathematics. A printed computer-algebra answer is not, by itself, a term that the proof

*Independent researcher. atawfeek.math@gmail.com.

kernel can check. Conversely, reimplementing a competitive computer algebra system inside a proof assistant is a much larger task than checking the evidence produced by one.

M2Lean therefore exchanges *certificates*, not execution traces or assertions of success. For example, membership of $x^3 + y^3$ in $(x + y, x - y) \subseteq \mathbb{Q}[x, y]$ is witnessed by the identity

$$x^3 + y^3 = \frac{1}{2}(x^2 + y^2)(x + y) + \frac{1}{2}(x^2 - y^2)(x - y),$$

which is checked by normalization. More elaborate claims carry cofactors, division quotients, or S-polynomial representations of the same general form. The potentially expensive search remains in *Macaulay2*; checking reduces to polynomial arithmetic and monomial comparisons. This asymmetry is useful because ideal membership is EXPSPACE-complete over $\mathbb{Q}$ in general [May89], whereas a proposed cofactor identity is directly verifiable.

There are two distinct verification paths. In the runtime path, `m2lean-check` parses a protocol document and executes compiled Boolean checkers. In the theorem path, certificate literals are elaborated into a *Lean* theorem file, the same checks are evaluated by kernel reduction, and formal soundness theorems are applied. Only the second path produces a proof term for the particular claim. This distinction is central to the trust model in §3 and is retained throughout the paper.

The current scope is deliberately narrower than the range of objects that *Macaulay2* can compute. It covers exact polynomial algebra over $\mathbb{Q}$ and $\mathbb{F}_p$, two monomial orders, ideals, matrices, graded free modules, and seven claim kinds. A chain-complex claim certifies only that consecutive differentials compose to zero; it does not certify exactness, minimality, or Betti numbers. Such boundaries are part of the interface rather than implicit assumptions.

Contributions. This paper describes a prototype with the following components.

1. A versioned protocol and *Macaulay2* package for exporting mathematical objects, evidence, and non-semantic provenance, together with an in-session command that invokes the standalone verifier (§4.1–§4.2).
2. A generic sparse polynomial model, executable checkers, and soundness theorems connecting accepted certificates to `mathlib` objects over $\mathbb{Q}$ and prime fields (§4.3).
3. A bridge-specific formal development for GRevLex Gröbner and non-membership certificates, including a `MonomialOrder.degRevLex` instance and agreement lemmas between the protocol comparator and the abstract order (§5).
4. Two consumption interfaces: a standalone runtime verifier and a prototype by `macaulay2` tactic that constructs a kernel-checked ideal-membership proof from a live *Macaulay2* response (§6).
5. Case studies and an acceptance/rejection evaluation comprising 15 valid and 20 invalid documents, together with theorem examples for the twisted cubic, the Jacobian map, graph colouring, and separability (§7.1–§8).

The evaluated artifact has the following immutable identity:

release	M2Lean 0.2.0, annotated tag v0.2.0
source commit	49f18427957e3a7960971d43c14ea4ce6991e472
Software Heritage release	swh:1:rel:b14fe2261b9eca1a816b9590ee6edda3d091feb2

The GitHub release supplies the raw benchmark record, deterministic source archive, checksum file, and a post-tag identity record that binds the first three to the evaluated commit [Taw26a, Taw26b].

Ordinary Software Heritage Git ingestion preserves the source and tag, not those separately attached release assets.

2 Related work

Skeptical combination of proof assistants and computer algebra goes back to Harrison and Théry’s HOL–Maple bridge [HT98], which established the pattern used here: let an external system search for an object, then re-check a compact witness in the prover. Delahaye and Mayero connected Coq to Maple for field simplification [DM05], and Kaliszyk and Wiedijk implemented certified computer algebra on top of HOL Light [KW07]. CoqEAL instead refines abstract algebraic structures to efficient executable representations within the prover [CDM13]. Certificate reconstruction is also standard in SMT solving [BBFF20] and sums-of-squares automation [Har07].

There are several direct predecessors in the Lean ecosystem. The Lean–Mathematica interface of Lewis and Wu exchanges data in both directions and uses Mathematica as an untrusted source of proof-search guidance [LW22]. The `linear_combination` tactic checks explicit cofactor identities [Gol22]. The now unsupported `polyrith` tactic historically obtained such identities from an external Sage service and reconstructed a Lean proof; the pinned `mathlib` release instead offers an internal `grobner` tactic for polynomial consequences [Bha22]. Most closely related, Shen, Guo, Liu, and Zhi provide certificate-based tactics that call SageMath or SymPy and support remainder certification, Gröbner bases, ideal equality, ideal membership and non-membership, and radical membership over lexicographically ordered rational polynomial rings [SGLZ26]. M2Lean is therefore not the first external-CAS or Gröbner tactic for Lean. Its different focus is a Macaulay2-facing, standalone and versioned document protocol that also supports M2-initiated workflows, runtime reports, prime fields, GRevLex, matrices, and complexes. Its `by macaulay2` tactic is a small demonstration of that protocol rather than a replacement for the broader tactic suite of Shen et al.

On the formalization side, Buchberger’s algorithm was verified in Coq by Théry [Thé01], and Isabelle/HOL has an extensive Gröbner development [IM16]. Guo, Shen, Liu, and Zhi have since formalized Gröbner basis theory in Lean 4, including division, Buchberger’s criterion, and reduced bases, with parts of the infrastructure integrated into `mathlib` [GSLZ26]. The project-local development in §5 is narrower: it targets the standard-representation certificate emitted by M2Lean, constructs the GRevLex order needed for *Macaulay2*’s default convention, and proves agreement between that abstract order and the executable list representation. The contribution claimed here is this connection to the protocol checker, not priority for Buchberger formalization in Lean.

The natural certificates used below—cofactors, standard representations, and division data—are classical [CLO15, BW93]. What requires engineering is fixing their exact cross-system representation and stating precisely when a runtime check has, or has not, been promoted to a kernel theorem.

3 Architecture and assurance

3.1 Layers and verification paths

layer	realization
interchange representation	<code>protocol/SPEC.md</code> , JSON schema, fixtures
semantics	<i>Lean</i> : <code>sparse model</code> $\rightarrow$ <code>MvPolynomial (Fin n) α</code>
claims and certificates	<i>Lean</i> checkers + soundness theorems
integrations	<i>Macaulay2</i> package; <code>m2lean-check</code> ; <i>Lean</i> tactic/theorems

A protocol document can be consumed in two ways. The standalone *runtime path* parses and structurally validates the document, runs one compiled Boolean checker per claim, and emits a report. This path is convenient inside a *Macaulay2* session and for batch validation, but the report is not a proof term. The *theorem path* embeds selected certificate data as *Lean* literals, establishes the concrete Boolean acceptance equation with `decide +kernel`, and applies a general soundness theorem. The result is a theorem of the corresponding *mathlib* proposition. Figure 1 shows the latter path for the twisted-cubic example.

The version-controlled `json2lean.py` generator used by the examples is not verified. Kernel checking protects the theorem that is generated, but it does not by itself prove that the literals are a faithful transcription of a particular JSON file. The checked-in release pipeline checks the raw-byte digests of all eleven shipped JSON documents and four derived *Lean* data modules, builds the runtime verifier, and regenerates those artifacts in the environment resolved for that run. It then checks their digests again against the committed SHA-256 ledger and builds the theorem modules. A successful run establishes byte-for-byte replay in that environment and detects drift; it does not establish environment-independent determinism or semantically verify the generator.

Throughout the paper, code listings are color-coded by their owning system: **red** for *Macaulay2* (computation and evidence generation), **amber** for protocol documents (the data crossing the boundary), and **blue** for *Lean* artifacts. These textual labels, not color alone, carry the distinction.

3.2 Trust boundary

For a theorem produced by the theorem path, the logical trusted base is the *Lean* kernel together with the declared axioms. The formal definitions and theorem statements determine what has actually been proved, so matching them to the intended polynomial rings, variable order, and geometric interpretation is a separate semantic-review obligation. *mathlib* and the M2Lean proofs are checked by the kernel; they are not additional unverified proof engines.

The *Macaulay2* engine, M2Lean’s *Macaulay2* package, serialization, transport, the JSON-to-*Lean* generator, and generated syntax before kernel acceptance are outside this logical boundary. A defect in any of them may cause rejection or may supply different data; assuming the checker soundness theorem and kernel are correct, it cannot furnish a proof of a false formal proposition relative to the declared axioms. This statement does not apply to a CLI report in isolation: relying on that report also relies operationally on the parser, compiler, executable, and runtime. As with any mechanized proof, performing a real check additionally assumes a correctly executed kernel binary and ordinary hardware and operating-system operation. The logical result is therefore narrower than a claim that “no implementation is trusted.”

The theorem examples avoid `native_decide`; their certificate equalities are reduced by the kernel with `decide +kernel`. The axiom audit in §8 reports the declarations on which the resulting theorems depend and checks that no project-specific axioms were introduced.

3.3 Report labels and theorem realization

The protocol defines four report labels. They classify the operation performed and whether a general soundness theorem exists; they do not say that a proof term was constructed for the particular report.

report label	meaning
<code>transported</code>	object parsed and structurally validated; no claim checked
<code>reproduced</code>	computation re-run and matched; no certificate checked
<code>checked</code>	an executable checker accepted; no formal soundness theorem is supplied
<code>proved</code>	an executable checker accepted and a general kernel-checked theorem proves that this checker is sound

Accepted claims from the implemented verifiers currently emit only `checked` or `proved`; rejected results carry `none`. The `transported` and `reproduced` labels are reserved by the protocol but are not emitted for an implemented claim kind in this release.

The name `proved` is retained because it is the current wire-format label, but it is best read as *soundness-formalized*. A fifth status, not encoded in runtime reports, is a *kernel-instantiated theorem*: a theorem file has established the concrete check and applied the soundness result. In the current implementation, polynomial identity, ideal membership, span inclusion, and chain-complex composition have soundness-formalized checkers. Gröbner and non-membership checkers do so for GRevLex; their Lex instances and the graded-complex checker remain executable-only. Claims are evaluated in the declared coefficient field, either $\mathbb{Q}$ or $\mathbb{ZMod}\ p$ after a primality check.

4 Protocol and implementation

4.1 The interchange protocol

The current protocol version is 0.2.0; consumers reject any other version string. It deliberately covers a small, exactly specified fragment: coefficients in $\mathbb{Q}$ (reduced fractions of decimal strings; JSON numbers are not used for coefficients or field scalars, although exponents and indices are nonnegative JSON integers) or $\mathbb{F}_p$ (p verified prime by the consumer); multivariate polynomial rings with positional variable identity and one of two normatively defined monomial orders (`Lex` and `GRevLex`, *Macaulay2*'s default); sparse polynomials in canonical form (nonzero reduced coefficients, exact arity, strictly decreasing terms); ideals; matrices; and graded free modules $\bigoplus_j R(-d_j)$. Exact arity is a wire-format invariant: every serialized term has one exponent per declared variable. The checker-level sparse type below is intentionally more permissive.

Except for the two raw sides of a `PolynomialIdentity` claim, the protocol requires canonical polynomials, making equality of valid encodings structural. The validator rejects, for example, object and certificate terms that are misordered for the declared order, including the element and cofactors of a membership claim. It also rejects duplicate identifiers and collisions between

object and claim identifiers. Normative order definitions and fixtures distinguish the incompatible textbook conventions commonly called graded reverse lexicographic.

A *claim* is a proposition about previously introduced objects plus evidence addressed to a specific checker:

claim kind	proposition	evidence
PolynomialIdentity	$lhs = rhs$	none (normalization)
IdealMembership	$f \in (g_1, \dots, g_k)$	cofactors c_i with $f = \sum c_i g_i$
SpanInclusion	$I \subseteq J$	a cofactor row per generator
GroebnerBasis	G is a GB of I	span witnesses both ways + S-pair standard representations
NonMembership	$f \notin I$	division data vs. a certified GB: $f = \sum q_k b_k + r, r \neq 0$ reduced
ChainComplex	$d_i \circ d_{i+1} = 0$	none (matrix arithmetic)
GradedComplex	the above, graded	none (homogeneity checks)

Documents also carry non-semantic provenance fields such as producer versions and algorithm names. The parser validates the required field types and copies the parsed JSON value unchanged as `inputProvenance` in the output report. Provenance remains informational and does not influence mathematical acceptance: for example, `deterministic: false` describes certificate discovery, not the independent verification of the supplied evidence. CLI diagnostics distinguish parse, structural, and certificate-verification failures; top-level parse and structural failures are written to standard error rather than represented as JSON reports.

The most structured evidence format is the Gröbner one. For every pair $i < j$ of basis elements, including pairs with coprime leading monomials, the producer must supply quotients $q_1, \dots, q_r$ such that

$$S(b_i, b_j) = \sum_k q_k b_k, \quad \text{lm}(q_k b_k) \preceq \text{lm}(S(b_i, b_j)) \text{ whenever } q_k \neq 0,$$

i.e. a *standard representation*. Requiring coprime pairs keeps the soundness proof within the standard-representation criterion, without an appeal to Buchberger’s first criterion. By Buchberger’s criterion in standard-representation form [BW93, Thm. 5.64], verifying these identities and bounds (plus the two span inclusions relating G to the input generators) establishes that G is a Gröbner basis. The checker does not run Buchberger’s search; it checks polynomial identities and monomial comparisons. For GRevLex, the connection from this executable check to the abstract criterion is a kernel-checked theorem (§5); for Lex it is not yet formalized.

Non-membership certificates depend on positive ones. Each such claim references a Gröbner-basis claim in the same document and supplies division data with a nonzero remainder no term of which is divisible by a basis leading monomial. A reduced nonzero remainder refutes membership only when the divisor family is a Gröbner basis, so this checker is soundness-formalized for GRevLex and remains executable-only for Lex.

4.2 The Macaulay2 bridge package

The package `m2/M2Lean.m2` exports objects and generates certificates using public *Macaulay2* interfaces. From the user’s point of view, encoding mathematics for the bridge is ordinary *Macaulay2*: one builds rings and ideals as usual, then attaches claims to a document and writes it out. A complete session certifying a membership looks like this:

```
loadPackage("M2Lean", FileName => "m2/M2Lean.m2");
R = QQ[x,y];
I = ideal(x+y, x-y); -- the mathematics, as usual
D = newM2LeanDocument "session-1";
membershipClaim(D, "mem1", x^3 + y^3, I); -- computes the cofactors
writeM2LeanDocument(D, "session-1.json"); -- crosses the boundary
```

The loop closes without leaving the session:

```
verifyWithLean D
-- m2lean-check report for document session-1:
-- mem1: accepted [proved] element lies in the ideal
```

`verifyWithLean` writes the document to a temporary file, spawns the Lean verifier as a subprocess, parses the report, and displays colored per-claim verdicts (green **proved**, amber **checked**, red rejections), returning the parsed report for programmatic use. Here **proved** is the runtime label *soundness-formalized*; the command does not construct a theorem for `mem1`.

`membershipClaim` divides the element against the ideal’s Gröbner basis, extracts cofactors, re-checks the identity, and serializes everything—generators, element, evidence, monomial order, provenance—in the canonical forms of §4.1. The evidence is subsequently checked, so its production may use ordinary *Macaulay2* machinery: membership cofactors come from matrix division `f // gens I` against *Macaulay2*’s internal Gröbner engine, and basis conversion witnesses likewise. For S-pair evidence, *Macaulay2*’s engine does not expose its reduction traces, so the package re-derives the quotients with an explicit textbook division algorithm (twenty lines of *Macaulay2* code) and exports the resulting standard-representation data. The consumer independently checks both the identity and the required leading-monomial bounds.

Macaulay2 keeps polynomials sorted in the ring’s monomial order. Conformance tests compare that ordering with the protocol definition (for example, $x + y^2z$ serializes as $[x, y^2z]$ under `Lex` and $[y^2z, x]$ under `GRevLex`). These are regression tests rather than a proof about *Macaulay2* internals; a disagreement on a multi-term polynomial causes structural rejection. The direct-load, package-check, installation, and fresh-load workflows were exercised with *Macaulay2* versions 1.24.11 and 1.26.06; the measurements in §8 use 1.24.11.

4.3 The Lean library

4.3.1 An executable model and its interpretation

The computational heart is a deliberately small sparse model, parameterized over the coefficient field (instantiated at $\mathbb{Q}$ for `RationalField` documents and `ZMod p` for `PrimeField` documents, after the verifier confirms primality and synthesizes the field instance):

```
structure STerm (α : Type*) where
  coeff : α
  exps : List Nat -- positional; entries beyond the end are 0
abbrev SPoly (α : Type*) := List (STerm α)
```

At this internal level, missing exponent entries are interpreted as zero and normalization trims trailing zeros. Protocol validation nevertheless requires the exact wire arity stated above. The model provides executable normalization (sort by a fixed total comparator, then merge equal exponent vectors and drop zeros, with trailing-zero trimming so that paddings of the same monomial coincide), ring operations on raw term lists, and the two protocol monomial orders.

The normalization routines use structural recursion because the well-founded formulations tried during development did not reduce suitably when the checks were evaluated inside the kernel.

The semantics is a coefficient-generic function $\text{toMv } n : \text{SPoly } \alpha \rightarrow \text{MvPolynomial } (\text{Fin } n) \ \alpha$ interpreting a term list as a sum of monomials, together with a lemma suite proved once and for all: toMv commutes with concatenation, term-wise product, permutation, merging, and normalization. This yields soundness for two core executable predicates used throughout the certificate checkers:

```
theorem polyEq_sound (h : polyEq p q = true) : toMv n p = toMv n q
theorem polyIsZero_sound (h : polyIsZero p = true) : toMv n p = 0
```

4.3.2 Checkers and soundness theorems

Checkers are Boolean functions on plain sparse data—independent of the protocol AST, so the soundness theorems apply to them directly. Representative statements (all **sorry**-free, all checked by the kernel):

```
theorem checkMembership_sound
  (h : checkMembership f gs cs = true) :
  toMv n f ∈ Ideal.span {p | p ∈ gs.map (toMv n)}

theorem checkSpanInclusion_sound
  (h : checkSpanInclusion src tgt rows = true) :
  spanOf n src ≤ spanOf n tgt

theorem checkComplexPair_sound
  (h : checkComplexPair r m c A B = true) :
  toMatrix n r m A * toMatrix n m c B = 0

theorem no_common_zero
  (h : checkMembership onePoly gs cs = true)
  {K : Type*} [CommRing K] [Nontrivial K] [Algebra αK]
  (pt : Fin n → K)
  (hv : ∀ g ∈ gs, aeval pt (toMv n g) = 0) : False
```

The last theorem is the abstract half of the twisted-cubic example: a certificate $1 = \sum c_i g_i$ makes the span the unit ideal and rules out a simultaneous zero in any nontrivial commutative algebra over the coefficient field.

The Gröbner checker `checkGroebner` verifies S-pair coverage, each standard representation with its leading-monomial side conditions, and both span inclusions. Section 5 connects acceptance to the abstract Gröbner property for GRevLex. The analogous connection for Lex remains unformalized.

4.3.3 Kernel-checked certificate consumption

The CLI accepts documents at runtime, but selected example *theorems* embed certificate data as literals generated by the version-controlled, unverified `json2lean.py` script and verify the check by kernel reduction:

```
theorem certificate_checks :
  checkMembership onePoly gens cofactors = true := by decide +kernel
```

Two implementation details affected kernel evaluation. First, the well-founded recursive formulations tried for the merge step did not reduce in these checks, so the implementation uses a structural `foldr`. Second, plain `decide` did not reduce the relevant `Rat` computations in this development, whereas `decide +kernel` did. Applying the corresponding soundness theorem to this equality, rather than the CLI report, produces the mathematical theorem.

5 Formal soundness for the GRevLex tier

For GRevLex, acceptance of the Gröbner checker implies the abstract Gröbner property by a kernel-checked proof. This connection uses three project-local components. It overlaps mathematically with the broader Lean development of Guo et al. [GSLZ26], but is organized around M2Lean’s executable certificate format.

Buchberger’s criterion. We prove the standard-representation form of Buchberger’s criterion [Buc70, BW93] over an arbitrary `mathlib MonomialOrder` and field:

```
theorem buchberger_criterion
  (hb : ∀ i, b i ≠ 0)
  (hS : ∀ i j, HasStdRep m b (sPoly m (b i) (b j)))
  (hf : f ∈ Ideal.span (Set.range b)) (hf0 : f ≠ 0) :
  ∃ i, m.degree (b i) ≤ m.degree f
```

Here the displayed $\leq$ is componentwise comparison of exponent vectors, equivalently divisibility of the leading monomial of b_i into that of f ; it is not comparison in the monomial order. The proof is the classical one, mechanized: pick a representation of f minimizing (by well-foundedness of the order) the largest product degree δ ; if δ exceeds $\deg f$, the top-degree layer cancels and is rewritten through S-polynomial standard representations—a cancellation lemma proved by strong induction on the size of the layer—yielding a representation with smaller δ . The statement uses `mathlib’s MonomialOrder/degree` interface so that it can interoperate with library polynomials.

The degree-reverse-lexicographic order. The pinned `mathlib` release (v4.32.0) provides `lex` and `degLex`, but not *Macaulay2’s* default order. We construct `MonomialOrder.degRevLex` as a type synonym ordered by total degree, ties broken by the *order dual* of `mathlib’s` colexicographic comparison. One subtlety is worth recording: unlike `degLex`, whose tie-break order is itself well-founded, plain reverse-lexicographic comparison is *not* $(1 \succ x_0 \succ x_0^2 \succ \dots)$, so well-foundedness cannot be inherited componentwise. It holds anyway because degrees are non-increasing along a descending chain and, over finitely many variables, only finitely many exponent vectors exist below a given degree — which is why `degRevLex` requires `[Finite σ]`, a hypothesis `degLex` does not need.

Connecting the checker to the abstract order. Connecting the executable checker to the abstract criterion requires agreement lemmas—the comparator on exponent lists computes exactly the `degRevLex` relation on interpreted monomials—and a *lead bridge*: the head of the sorted normalization is the abstract leading term. The latter would need a formal theory of sorting-with-merging; instead, the checker was refined to certify the facts the proof needs. A `leadOK` check asserts, decidably, that the head strictly dominates every other term; soundness

then consumes the check rather than a meta-theorem about the sorting algorithm. The resulting statements are:

```

theorem checkGroebner_sound
  (h : checkGroebner n .grevlex gens basis bc gc sps = true) :
  spanOf n gens = spanOf n basis ∧
  ∀f ∈ spanOf n basis, f ≠ 0 →
    ∃b ∈ basis, (grevlexOrder n).degree (toMv n b) ≤
      (grevlexOrder n).degree f

theorem checkNonMembership_sound
  (hgb : checkGroebner n .grevlex gens basis bc gc sps = true)
  (hnm : checkNonMembership n .grevlex basis f quotes r = true) :
  toMv n f ∉ spanOf n basis

```

Both hold over any coefficient field and therefore apply to $\mathbb{F}_p$ documents. The second uses the first: division data with a nonzero reduced remainder refutes membership only against a soundness-formalized Gröbner basis. Certificates over *Lex* rings remain at *checked* until corresponding order-agreement lemmas are proved; the runtime report computes its label per order.

6 The `by macaulay2` tactic

The protocol can also support a Lean-initiated workflow. The current prototype exposes one such tactic call:

```

def f : SPoly ℚ := [⟨1, [3, 0]⟩, ⟨1, [0, 3]⟩] -- x3 + y3
def gs : List (SPoly ℚ) :=
  [[⟨1, [1, 0]⟩, ⟨1, [0, 1]⟩], [⟨1, [1, 0]⟩, ⟨-1, [0, 1]⟩]]

example : toMv 2 f ∈ spanOf 2 gs := by macaulay2

```

The tactic reflects the goal’s sparse data, writes a *Macaulay2* session requesting a membership certificate, runs `M2 -script` as a subprocess, parses the returned protocol document with the `M2Lean` parser, embeds the cofactors as literals, and closes the goal with `checkMembership_sound` applied to a kernel-checked run of the certificate checker. The answer enters the proof as data. If the subprocess, parse, or transcription supplies data that do not establish the fixed goal, kernel checking fails; under the trust assumptions of §3, the external process cannot authorize a false goal. The release gate explicitly builds `M2Lean.Examples.TacticDemo` after the main library target. This exercises a live *Lean-to-Macaulay2* round trip when the module is elaborated. The clean-cache release job guarantees that elaboration; an ordinary cache-restored CI run may reuse an unchanged demo module. This prototype handles only ground goals already expressed in `M2Lean`’s sparse representation. The tactics of Shen et al. support a broader set of reflected polynomial goals [SGLZ26]; `M2Lean`’s tactic is included to demonstrate reuse of the document protocol.

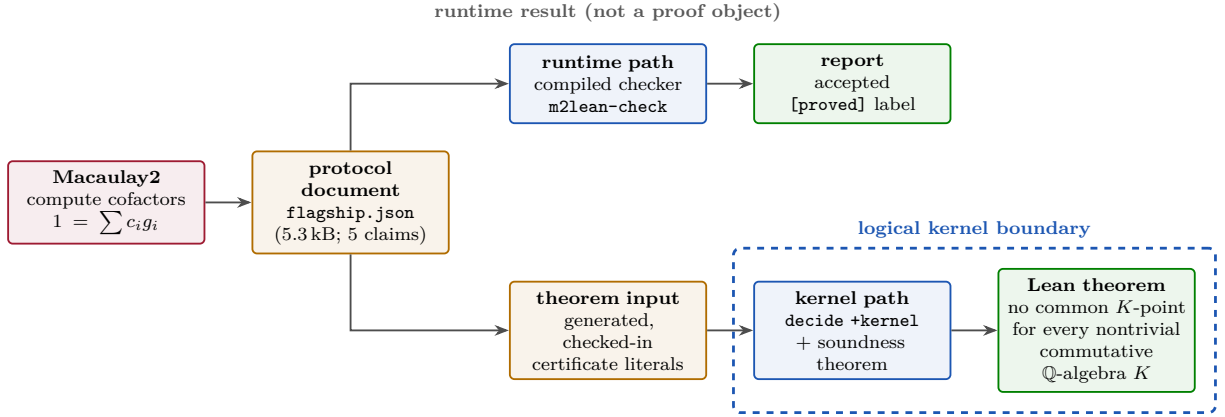


Figure 1: Two consumers of the same certificate document. The upper lane produces a runtime report; **proved** labels the checker’s formal soundness. In the lower lane the particular certificate is re-evaluated inside the dashed kernel boundary and the soundness theorem is applied, producing a theorem. The *Macaulay2* producer, JSON, code generator, and generated literals before kernel acceptance are outside that logical boundary.

7 Case studies

7.1 Twisted-cubic disjointness

This standard example has a certificate short enough for direct inspection and replay in continuous integration (cf. [Eis95]). It demonstrates the end-to-end semantics; it is not intended as a difficult search problem. Let $R = \mathbb{Q}[x, y, z, w]$ and let

$$I = (xz - y^2, xw - yz, yw - z^2)$$

be the ideal of 2×2 minors of $\begin{pmatrix} x & y & z \\ y & z & w \end{pmatrix}$, cutting out the cone over the twisted cubic curve. Let $L = V(y - 1, z, w)$ be the line $\{(t, 1, 0, 0)\}$. Figure 1 traces the artifacts through the pipeline; the remainder of this section walks the same path in detail.

Step 1: computation (*Macaulay2*). The mathematics is encoded in ordinary *Macaulay2* syntax:

```

R = QQ[x,y,z,w];
I = minors(2, matrix{{x,y,z},{y,z,w}}); -- the twisted cubic cone
J = ideal(y - 1, z, w); -- the line {(t,1,0,0)}
assert(I + J == 1); -- unit ideal

```

The assertion can also be verified directly. Its role is to provide a transparent query whose algebraic and geometric interpretations can be followed through the full pipeline.

Step 2: certificate (*Macaulay2*). Five claims are attached to a document and exported:

```

D = newM2LeanDocument "flagship";
unitIdealClaim(D, "unit1", I + J); -- 1 = sum c_i g_i
gbClaim(D, "gb1", I); -- Buchberger evidence
C = res comodule I;
nonMembershipClaim(D, "nm1", x, I); -- x is nonzero modulo I
gradedComplexClaim(D, "res1", {C.dd_1, C.dd_2});
chainComplexClaim(D, "cx1", {C.dd_1, C.dd_2});
writeM2LeanDocument(D, "flagship.json");

```

The membership certificate for 1 carries cofactors—in this instance

$$1 = (-1) \cdot (xz - y^2) + (-y - 1) \cdot (y - 1) + x \cdot z,$$

a directly checkable identity. It proves $I + J = R$. Geometrically, $V(I) \cap V(J) = V(I + J)$ has coordinate ring $R/(I + J) = 0$, so the closed subschemes have empty scheme-theoretic intersection. Equivalently, evaluating the identity at a simultaneous zero in a nontrivial commutative $\mathbb{Q}$ -algebra would give $1 = 0$.

The document also contains a Gröbner certificate for I (three S-pairs with standard representations) and matrices that *Macaulay2* reports as the minimal Eagon–Northcott resolution [EN62]

$$0 \rightarrow R(-3)^2 \xrightarrow{d_2} R(-2)^3 \xrightarrow{d_1} R \rightarrow R/I \rightarrow 0.$$

The runtime verifier checks that the displayed matrices form a complex and accepts their declared degrees; it does not verify exactness or minimality, so it does not certify the Betti numbers. The complete JSON document is 5,310 bytes (5.3 decimal kB).

Step 3: verification (*Lean*). `m2lean-check` accepts all five claims. The last column contains runtime report labels in the sense of §3:

claim	runtime result	report label
unit1 (membership of 1 in $I + J$)	accepted	proved
gb1 (Gröbner basis of I)	accepted	proved
nm1 ($x \notin I$)	accepted	proved
res1 (composition and degree checks)	accepted	checked
cx1 ($d_1 d_2 = 0$)	accepted	proved

Step 4: theorem realization (*Lean*). The example file re-checks the unit-ideal certificate and the chain-complex computation in the kernel and applies their soundness theorems, yielding (with type annotations elided):

```

theorem twisted_cubic_cone_misses_line
  {K : Type*} [CommRing K] [Nontrivial K] [Algebra ℚK]
  (p : Fin 4 → K)
  (h1 : aeval p (X 0 * X 2 - X 1 ^ 2) = 0)
  (h2 : aeval p (X 0 * X 3 - X 1 * X 2) = 0)
  (h3 : aeval p (X 1 * X 3 - X 2 ^ 2) = 0)
  (h4 : aeval p (X 1 - 1) = 0)
  (h5 : aeval p (X 2) = 0)
  (h6 : aeval p (X 3) = 0) : False

theorem resolution_composes_to_zero :
  toMatrix 4 1 3 cx1_d1 * toMatrix 4 3 2 cx1_d2 = 0

```

The first statement says that there is no common K -valued point for any nontrivial commutative $\mathbb{Q}$ -algebra K . The finite identity over $\mathbb{Q}$ is interpreted uniformly through the soundness theorem. The example’s purpose is this explicit semantic passage, not the difficulty of finding these particular cofactors. An axiom audit reports that the declarations used are `propext`, `Classical.choice`, and `Quot.sound`—no `sorry`, no project-specific axioms, no `native_decide`.

7.2 The 2026 Alpöge–Fable counterexample

The Jacobian conjecture, introduced by Keller [Kel39], asserts that a polynomial map $F : \mathbb{C}^n \rightarrow \mathbb{C}^n$ with nonzero constant Jacobian determinant has a polynomial inverse. In July 2026, Alpöge announced the following dimension-three counterexample, crediting Akhil Mathew for posing the question and the AI system Claude Fable 5 for the construction [Alp26]. We therefore call it the Alpöge–Fable map. Contemporaneous Isabelle/HOL and independent Lean developments also formalized the determinant and collision, with the Isabelle entry proving the implication for the classical conjecture [RHdQ26, NG26]. M2Lean makes no discovery claim for the map; the present case study records the narrower facts checked by this artifact and how those checks enter its *Lean* theorem.

The mathematical verification separates into a constant-Jacobian identity and a collision of rational points. The map is

$$\begin{aligned} F(x, y, z) = & ((1 + xy)^3 z + y^2(1 + xy)(4 + 3xy), \\ & y + 3x(1 + xy)^2 z + 3xy^2(4 + 3xy), \\ & 2x - 3x^2 y - x^3 z). \end{aligned}$$

Protocol checks. The *Macaulay2* script forms the Jacobian, computes its determinant, and exports the equality with -2 as a polynomial-identity claim:

```
R = QQ[x,y,z];
F = {(1+x*y)^3*z + y^2*(1+x*y)*(4+3*x*y),
      y + 3*x*(1+x*y)^2*z + 3*x*y^2*(4+3*x*y),
      2*x - 3*x^2*y - x^3*z};
Jac = matrix table(3,3,(i,j) -> diff({x,y,z}#j, F#i));
assert(det Jac == -2); -- nonzero constant Jacobian
polynomialIdentityClaim(D, "detJac", det Jac, -2_R);
```

At the protocol level, `detJac` checks only that the serialized polynomial equals -2 ; it does not certify that this polynomial was obtained by differentiating the displayed map. The semantic link to F is established separately in the theorem file by defining the Jacobian with `pderiv` and proving `jacF_det`.

The three distinct points $P_0 = (0, 0, -\frac{1}{4})$, $P_1 = (1, -\frac{3}{2}, \frac{13}{2})$, and $P_2 = (-1, \frac{3}{2}, \frac{13}{2})$ all map to $(-\frac{1}{4}, 0, 0)$, so F is not injective and has no polynomial inverse. These points are supplied explicitly by the example script; this artifact does not document or certify the process by which they were found. Each component value $F_i(P) = v_i$ is exported as a membership in the point ideal $\mathfrak{m}_P = (x - a, y - b, z - c)$: the cofactor identity $F_i - v_i = A(x - a) + B(y - b) + C(z - c)$, evaluated at P , kills the ideal part and forces $F_i(P) = v_i$. Nine such membership claims (three points $\times$ three coordinates) together with the determinant identity form a 12,652-byte document (12.7 decimal kB). The runtime verifier accepts all ten claims with the `proved` report label.

Theorem realization and dependency ledger. The theorem file contains four logically distinct developments:

```

theorem jacF_det : jacF.det = -2

theorem F_not_injective :
  ¬Function.Injective (fun p : Fin 3 → ℚ => fun i => aeval p (F i))

theorem jacobian_conjecture_false : ¬JacobianConjecture 3

theorem polynomial_jacobian_conjecture_false :
  ¬PolynomialJacobianConjecture 3

```

The determinant theorem expands `pderiv` and closes the resulting identity with `ring`. Separately, `certificate_checks` re-evaluates all nine exported membership certificates in the kernel. The example applies `checkMembership_sound` to all nine in `collision_certified`; that conjunction is not used by the final non-injectivity proof. Instead, `F_not_injective` evaluates the displayed formula at P_1 and P_2 directly with `norm_num` and uses their unequal first coordinates. Thus the kernel-instantiated certificate checks and the final collision proof are parallel validation paths in the current file; it would be inaccurate to describe the latter as derived from all nine memberships.

The project-local definition `JacobianConjecture n` states the rational-point injectivity consequence. A second definition, `PolynomialJacobianConjecture n`, requires a two-sided inverse whose coordinates are polynomials over $\mathbb{Q}$ and whose inverse identities hold on every rational point. The file refutes both formulations: a left inverse alone would force injectivity, contradicting the displayed rational collision. These are statements over $\mathbb{Q}$, not definitionally the usual formulation over $\mathbb{C}$. Mathematically, the same rational collision persists after scalar extension and therefore also rules out a polynomial inverse over $\mathbb{C}$; that scalar-extension implication is not formalized in this file. The Lean file proves the dimension-three statements only; extension to higher dimensions by adjoining identity coordinates is likewise not formalized there. The axiom audit for these declarations reports `propext`, `Classical.choice`, and `Quot.sound`, with no project-specific axiom, `sorry`, or `native_decide`.

7.3 Graph non-colourability

Ideal-membership certificates can also establish infeasibility. For a graph G , following De Loera–Lee–Malkin–Margulies [DLLMM08], introduce one variable x_v per vertex, the relations $x_v^3 - 1$, and for every edge uv the relation $x_u^2 + x_u x_v + x_v^2$; write $\text{col}(G)$ for the ideal generated by these polynomials. Interpreting colours as the three cube roots of unity identifies a proper 3-colouring with a common zero of this system over $\mathbb{C}$. Consequently,

$$G \text{ is 3-colourable} \iff 1 \notin \text{col}(G),$$

and a Nullstellensatz certificate $1 = \sum_i c_i g_i$ is a proof of *non*-colourability.

`mathlib` contains Alon’s Combinatorial Nullstellensatz [Alo99]; the present example instead uses a unit-ideal witness to establish nonexistence and connects the result to `mathlib`’s graph-colourability predicate.

The kernel theorem. The vanishing lemmas are polymorphic over vertex sets `Fin n`, but the current Lean development specializes the serialized generators and graph-to-ideal correspondence

to the odd wheel W_5 obtained by adjoining a hub to C_5 , represented as a concrete graph on [Fin 6](#). It does not yet provide a generic graph-to-ideal construction. Given a proper colouring c , the proof constructs the point $x_v = \zeta^{c(v)}$, where ζ is a primitive cube root of unity in $\mathbb{C}$. A vertex relation vanishes because $(\zeta^{c(v)})^3 = 1$, an edge relation because $\zeta^{3c(u)} = \zeta^{3c(v)} = 1$ while $\zeta^{c(u)} \neq \zeta^{c(v)}$. The theorem then supplies this common zero to `no_common_zero`, contradicting the kernel-checked unit-ideal certificate:

```
theorem wheel5_not_three_colorable : ¬W.Colorable 3
```

The axiom audit reports the same three standard axioms as in the other theorem examples and no project-specific axiom or `sorry`.

Three concrete documents. The artifact also contains unit-ideal documents for K_4 and the Grötzsch graph, the Mycielskian of the 5-cycle [\[Myc55\]](#). The table reports total JSON size and end-to-end runtime-verifier wall time; each timing includes executable startup, parsing, validation, checking, and report construction.

graph	vertices	edges	document (kB)	runtime (ms)
K_4	4	6	4.5	31.0
W_5	6	10	8.0	35.9
Grötzsch	11	20	114.5	774.7

These three observations are a case study, not a scaling law. All three documents are accepted by `m2lean-check`; only W_5 is connected in the repository to a kernel-checked non-colourability theorem. For K_4 and the Grötzsch graph, `m2lean-check` accepts the unit-ideal certificates, but this release does not instantiate either document as a concrete Lean non-colourability theorem. The corresponding combinatorial implication is the classical encoding argument above.

8 Evaluation

The evaluation asks three bounded questions: whether valid example documents are accepted, whether selected adversarial certificate and format defects are rejected, and what the current end-to-end costs are on the included instances. It does not attempt to establish asymptotic or research-scale performance.

8.1 Acceptance and rejection tests

The script `scripts/run-checks.sh` regenerates eleven example documents with *Macaulay2*, checks those and four valid protocol fixtures, and expects rejection of 20 invalid fixtures. The invalid set combines mutations of generated-style certificates with deliberately handwritten structural and parsing cases; it should not be read as a statistical fault model. All 35 document expectations pass:

fixture or group	injected defect	observed result
15 valid documents	none	all accepted
six corrupt-* fixtures	false certificate evidence	verification rejection
five noncanonical-* fixtures	invalid canonical encoding	structural rejection
eight structure-* fixtures	invalid references, IDs, domain, order, or provenance	structural rejection
parse-truncated	truncated JSON	parse rejection

The harness asserts both the documented exit code and the diagnostic class (parse, structural, or verification). It also checks that a valid document marked **deterministic: false** is accepted with **proved** assurance and that its input provenance survives the report round trip. More systematic property-based mutation remains future evaluation work.

8.2 Measurement method and results

Measurements were taken on Debian GNU/Linux 13.5 under WSL 2. The recorded kernel was

6.18.33.1-microsoft-standard-WSL2.

The host had an AMD Ryzen 7 9800X3D (16 logical CPUs) and 31,651,472 KiB (30.2 GiB) RAM. The software versions were *Macaulay2* 1.24.11, *Lean* 4.32.0, and Lake 5.0.0. The pinned mathlib commit was

81a5d257c8e410db227a6665ed08f64fea08e997.

For each runtime-verifier row, the executable was run once to warm caches and then timed three times; the table reports the median wall time. The benchmark is sequential but was not CPU-pinned. Runtime-verifier measurements include process startup, JSON parsing, validation, checking, and report construction. Sizes are total JSON document sizes in decimal kilobytes. Peak RSS is the maximum recorded resident-set size. The benchmark driver retains every warm-up and timed sample, command, standard output and error, exit status, peak resident memory, environment, and source commit in schema-validated JSON.

document	claims	size (kB)	runtime (ms)	peak RSS (MiB)
flagship	5	5.3	36.1	70.5
jacobian	10	12.7	30.9	70.0
grotzsch	1	114.5	774.7	72.6
wheel5	1	8.0	35.9	70.7
primefield	3	2.3	36.0	70.4
coloring (K_4)	1	4.5	31.0	70.6
polynomial	4	2.5	30.8	70.8
galois	1	0.9	36.1	70.4
toric	1	1.1	36.0	70.8
stanley-reisner	2	1.2	30.9	70.8
symmetric	1	0.8	36.0	70.8

The single-run generation observations were:

script	ms	script	ms	script	ms
polynomial.m2	319	flagship.m2	352	jacobian.m2	328
coloring.m2	1,118	appendix.m2	347		

The colouring script emits two documents and the appendix script six. These figures include *Macaulay2* startup and file emission and are descriptive observations, not summary statistics.

The harness also times a copy of each document with its claims removed, as a parse-and-validation baseline. Subtracting this baseline is only a rough estimate: the process monitor polls at 5 ms intervals. The Grötsch document’s estimated 743.7 ms checking increment is clearly resolved at that granularity; the other estimated differences, from zero through 10.2 ms, are not interpreted quantitatively. In particular, the data do not support sub-millisecond checker-cost claims.

Kernel theorem elaboration is a different workload. After one warm-up, each of four checked-in theorem files was measured three times. These measurements cover whole-file elaboration, certificate reduction, and kernel checking, rather than only the certificate check:

theorem file	median wall time (s)	peak RSS (GiB)
twisted cubic	5.86	6.43
Jacobian	4.10	6.54
W_5 colouring	8.12	7.34
separability	3.24	6.38

The measurements show feasibility for the included instances; the eleven documents are too few and too heterogeneous to support a general scaling claim.

8.3 Reproducibility scope

The checked-in continuous-integration workflow first checks the recorded SHA-256 digests of the fifteen shipped generated artifacts and then builds the runtime verifier. It regenerates eleven JSON examples and four `*Data.lean` modules, checks their digests again against the committed ledger, applies the JSON Schema as a prefilter, executes all 35 acceptance and rejection expectations, builds the complete *Lean* library, and runs the axiom audit. Thus byte-for-byte regeneration, runtime validation, and theorem-module builds in the recorded environment are a release gate, subject to the unverified-generator qualification above.

The `v0.2.0` tag workflow completed both that gate and a no-cache clean-room build, then captured and validated the benchmark record. The record names the exact commit above, `git_dirty: false`, an empty porcelain status, and the SHA-256 digest of an empty tracked diff. The external post-tag record binds that benchmark, the deterministic source archive, and its checksum file to the annotated tag, and both JSON records pass their published schemas [Taw26a]. Software Heritage independently preserves the Git source and tag [Taw26b]. The immutable in-tree manifest intentionally leaves its post-tag identity fields null because inserting a commit’s own identity or the checksum of a post-tag record would create a self-reference.

9 Limitations and roadmap

The prototype has four main limitations. First, assurance is heterogeneous. GRevLex Gröbner and non-membership checks have soundness theorems, whereas their Lex instances and the graded-complex checker are executable-only. A `ChainComplex` claim proves only $d_i d_{i+1} = 0$; neither it nor the graded claim establishes exactness, minimality, cokernel identification, or Betti numbers.

Second, provenance is typed and retained but intentionally descriptive, not authenticated. It can aid reproduction but cannot establish who performed a computation or whether the recorded

environment is complete. The protocol also remains a deliberately narrow, closed vocabulary; new coefficient domains, object kinds, and probabilistic propositions require versioned extensions rather than ad hoc fields.

Third, the implementation eagerly parses JSON and embeds large certificate literals. A Gröbner certificate carries a standard representation for every S-pair, so certificate growth can dominate the computed answer. The current examples do not determine where memory or elaboration becomes prohibitive. Compact encodings, streaming runtime checks, and reflective kernel checkers require a separate evaluation before research-scale claims are warranted.

Fourth, theorem production is only partially automated. The `by macaulay2` prototype accepts ground membership goals already expressed in the sparse model, and the checked-in example theorems use an unverified JSON-to-Lean generator. The release pipeline now regenerates those literals and checks them against a committed hash ledger before building the theorem files, which controls artifact drift but does not verify the semantic correctness of the transformation or reflect arbitrary user goals.

Immediate work is therefore to formalize the Lex and graded tiers, broaden the tactic’s reflection layer, and extend protocol conformance and mutation testing. Exactness and minimality require new certificate families rather than stronger prose about the existing complex check. A subsequent benchmark suite should vary variables, generators, basis size, coefficient domain, and certificate density independently and report failures as well as successful runs.

10 Conclusion

M2Lean demonstrates a concrete certificate boundary between *Macaulay2* and *Lean* 4. The principal result is not that every accepted document is already a Lean theorem: the runtime verifier and the kernel theorem path deliberately provide different assurance. Rather, the same explicit evidence can support a convenient standalone check and, when embedded and connected to a soundness theorem, a kernel-checked mathematical statement. The GRevLex development, trust analysis, and case studies show that this pattern extends beyond elementary polynomial normalization while also making the present limits visible. The v0.2.0 source and annotated tag are publicly archived in Software Heritage. The associated GitHub release provides the validated benchmark, deterministic source archive, and checksum file, together with a record binding those three files to the evaluated commit. The clean release gate regenerates all named example documents and validates them at runtime; separately, it rebuilds the four checked-in Lean theorem modules and runs the axiom audit. These results concern the stated examples and environment; they do not support a general scaling claim.

Acknowledgements

AI-assisted tools were used during implementation review and editorial revision. The author reviewed the generated code and prose and remains responsible for the mathematical statements, experiments, and conclusions. The author thanks the Macaulay2 and *Lean/mathlib* communities, whose public interfaces and libraries made this interoperability experiment possible.

A Complete protocol document

The following is the valid fixture `membership.json`, abridged only in whitespace. The claim certifies $2x \in (x + y, x - y)$ in $\mathbb{Q}[x, y]$ with cofactors $(1, 1)$.

```
{ "m2leanVersion": "0.2.0",
  "documentId": "fixture-membership",
  "objects": [
    { "id": "QQ", "kind": "RationalField" },
    { "id": "R", "kind": "PolynomialRing", "coefficients": "QQ",
      "variables": ["x", "y"], "monomialOrder": { "kind": "GRevLex" } },
    { "id": "I", "kind": "Ideal", "ring": "R", "generators": [
      { "terms": [
        { "coefficient": { "num": "1", "den": "1"}, "exponents": [1, 0] },
        { "coefficient": { "num": "1", "den": "1"}, "exponents": [0, 1] } ] },
      { "terms": [
        { "coefficient": { "num": "1", "den": "1"}, "exponents": [1, 0] },
        { "coefficient": { "num": "-1", "den": "1"}, "exponents": [0, 1] } ] } ] } ],
  "claims": [
    { "id": "c1", "kind": "IdealMembership", "ideal": "I",
      "element": { "terms": [
        { "coefficient": { "num": "2", "den": "1"}, "exponents": [1, 0] } ] },
      "evidence": { "cofactors": [
        { "terms": [ { "coefficient": { "num": "1", "den": "1"},
          "exponents": [0, 0] } ] },
        { "terms": [ { "coefficient": { "num": "1", "den": "1"},
          "exponents": [0, 0] } ] } ] } ],
      "provenance": { "producer": "handwritten", "producerVersion": "n/a",
        "packageVersion": "0.2.0", "deterministic": true } } ] }
```

The corresponding verification report:

```
{ "m2leanVersion": "0.2.0",
  "documentId": "fixture-membership-report",
  "reportFor": "fixture-membership",
  "results": [
    { "claim": "c1", "status": "accepted", "assurance": "proved",
      "message": "element lies in the ideal" } ],
  "inputProvenance": { "producer": "handwritten",
    "producerVersion": "n/a",
    "packageVersion": "0.2.0",
    "deterministic": true },
  "provenance": { "producer": "m2lean-check (Lean 4)",
    "packageVersion": "0.2.0" } }
```

B Additional examples

Each of the six examples below is condensed to the *Macaulay2* computation (red), the certificate crossing the protocol boundary, and what the *Lean* runtime verifier accepts (blue). All six documents are generated by `examples/appendix/appendix.m2` and verified by `m2lean-check` in the test suite; the displayed identities are drawn from those documents. A **proved** runtime

label means the claim family has a soundness theorem, as in §3; only the Galois example in this appendix is instantiated as a kernel theorem in the current repository.

B.1 Invariant theory: Newton’s identity

Power sums are polynomials in elementary symmetric functions; the degree-3 instance is a small example handled as a `PolynomialIdentity` claim.

```
R = QQ[x,y,z];
polynomialIdentityClaim(D, "newton3",
  x^3+y^3+z^3, e1^3 - 3*e1*e2 + 3*e3);
```

$$p_3 = e_1^3 - 3e_1e_2 + 3e_3.$$

Runtime verifier: ✓ `accepted` [`proved`] — both raw sides normalize to the same canonical form. The general theorem `checkIdentity_sound` is available to promote an embedded check to equality in `MvPolynomial (Fin 3) ℚ`; this document remains at runtime level.

B.2 Toric ideals and algebraic statistics: Markov moves

For 2×3 contingency tables, the toric ideal of the independence model contains three 2×2 minors, but Markov chains often use only the *adjacent* moves $m_1 = p_{11}p_{22} - p_{12}p_{21}$ and $m_2 = p_{12}p_{23} - p_{13}p_{22}$ [DS98]. The following certificate records one saturation relation: after multiplication by the middle coordinate p_{22} , the long minor lies in the ideal of the adjacent moves.

```
S = QQ[p11,p12,p13,p21,p22,p23];
membershipClaim(D, "markov",
  p22*(p11*p23 - p13*p21), ideal(m1, m2));
```

$$p_{22} (p_{11}p_{23} - p_{13}p_{21}) = p_{23} m_1 + p_{21} m_2.$$

Runtime verifier: ✓ `accepted` [`proved`] — the cofactor identity is accepted. The general theorem `checkMembership_sound` can promote an embedded check to membership in $\text{span}(m_1, m_2)$; the displayed cofactors are (p_{23}, p_{21}) .

B.3 Galois theory: separability of Φ_8

A polynomial is separable iff it is coprime to its derivative; the Bézout witness is a unit-ideal certificate.

```
T = QQ[t]; f = t^4 + 1;
unitIdealClaim(D, "separable", ideal(f, diff(t, f)));
```

$$1 = 1 \cdot (t^4 + 1) + \left(-\frac{t}{4}\right) \cdot 4t^3.$$

Runtime verifier: ✓ `accepted` [`proved`] — this crossing is kernel-instantiated: the certificate is embedded and kernel-checked (`Examples/Galois.lean`), the Bézout identity is transferred to univariate polynomials, and the result is `mathlib`’s own separability notion, `phi8_separable`: the polynomial $X^4 + 1$ is separable in $\mathbb{Q}[X]$.

B.4 Combinatorics: K_4 is not 3-colourable

Encoding colours as cube roots of unity, a graph G is 3-colourable iff the system $\{x_i^3 - 1\} \cup \{x_i^2 + x_i x_j + x_j^2 : ij \in E(G)\}$ has a solution; infeasibility is certified by a Nullstellensatz witness [DLLMM08].

```
G = QQ[x1,x2,x3,x4];
U = ideal(vertexPolys | edgePolys); -- 4 + 6 generators
assert(U == 1); -- computation
unitIdealClaim(D, "k4not3col", U);
```

For example, the cofactor attached to $x_1^2 + x_1 x_2 + x_2^2$ is

$$\frac{2}{9}x_3^2x_4^2 + \frac{2}{9}x_3x_4^3 - \frac{4}{9}x_4^4 - \frac{1}{3}x_3 + \frac{1}{3}x_4.$$

Runtime verifier: ✓ accepted [proved] — the runtime checker accepts the identity $1 \in U$. The soundness theorem rules out a common zero when instantiated; together with the classical encoding, this implies that K_4 admits no proper 3-colouring. This document is not instantiated as a kernel theorem in the repository.

B.5 A graded complex for a Stanley–Reisner ideal

The Stanley–Reisner ideal of the 4-cycle is the complete intersection $I = (ac, bd)$. *Macaulay2* returns the matrices and twists of its standard Koszul resolution, whose Betti numbers are 1, 2, 1; the bridge checks only the properties stated below.

```
Q = QQ[a,b,c,d]; Isr = ideal(a*c, b*d);
C = res comodule Isr;
gradedComplexClaim(D, "koszul", {C.dd_1, C.dd_2});
chainComplexClaim(D, "koszulcx", {C.dd_1, C.dd_2});
```

$$0 \rightarrow R(-4) \xrightarrow{\begin{pmatrix} -bd \\ ac \end{pmatrix}} R(-2)^2 \xrightarrow{(ac \ bd)} R \rightarrow R/I \rightarrow 0.$$

Runtime verifier: the report accepts `koszulcx` [proved] and `koszul` [checked]. The first is backed by `checkComplexPair_sound`, which gives $d_1 d_2 = 0$ as a `mathlib` `Matrix` identity. The executable graded checker also accepts the matrix dimensions, declared twists, and homogeneity of every entry. Neither result certifies exactness, minimality, the cokernel, or the displayed Betti numbers.

B.6 Finite fields: the same machinery over $\mathbb{F}_{101}$

The coefficient field is generic: the same checkers and the same soundness theorems run over `ZMod p`.

```
F = ZZ/101[x,y]; Ip = ideal(x^2 + y^2, x*y - 1);
gbClaim(D, "gbp", Ip);
membershipClaim(D, "memp", x^3 + x*y^2, Ip);
nonMembershipClaim(D, "nmp", x, Ip,
  "groebnerClaim" => "gbp"); -- x is NOT in Ip
```

Runtime verifier: ✓ accepted [proved] — all three claims — the Gröbner basis, the membership, and the negative certificate — verify over `ZMod 101` at the `proved` level; the Buchberger

soundness theorem is field-generic, so finite-field certificates inherit it without a rational-coefficient specialization. These claims remain runtime results in this appendix.

References

- [Alo99] Noga Alon. Combinatorial Nullstellensatz. *Combinatorics, Probability and Computing*, 8(1–2):7–29, 1999.
- [Alp26] Levent Alpöge. Announcement of the Alpöge–Fable counterexample to the Jacobian conjecture. Announcement on X, https://x.com/_alpoge_/status/2079028340955197566, 2026. 20 July 2026; credits Akhil Mathew for the question and Claude Fable 5 for the construction; archival formal verifications cited separately.
- [BBFF20] Haniel Barbosa, Jasmin Christian Blanchette, Mathias Fleury, and Pascal Fontaine. Scalable fine-grained proofs for formula processing. *Journal of Automated Reasoning*, 64:485–510, 2020.
- [Bha22] Dhruv Bhatia. `polyrith`: a Lean tactic closing polynomial arithmetic goals via an external computer algebra system. `mathlib4`, https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Polyrith.html, 2022.
- [Buc70] Bruno Buchberger. Ein algorithmisches Kriterium für die Lösbarkeit eines algebraischen Gleichungssystems. *Aequationes Mathematicae*, 4:374–383, 1970.
- [BW93] Thomas Becker and Volker Weispfenning. *Gröbner Bases: A Computational Approach to Commutative Algebra*, volume 141 of *Graduate Texts in Mathematics*. Springer, 1993.
- [CDM13] Cyril Cohen, Maxime Dénès, and Anders Mörtberg. Refinements for free! In *Certified Programs and Proofs (CPP 2013)*, volume 8307 of *Lecture Notes in Computer Science*, pages 147–162. Springer, 2013.
- [CLO15] David A. Cox, John Little, and Donal O’Shea. *Ideals, Varieties, and Algorithms*. Undergraduate Texts in Mathematics. Springer, 4th edition, 2015.
- [DLLMM08] Jesús A. De Loera, Jon Lee, Peter N. Malkin, and Susan Margulies. Hilbert’s Nullstellensatz and an algorithm for proving combinatorial infeasibility. In *Proceedings of the 21st International Symposium on Symbolic and Algebraic Computation (ISSAC 2008)*, pages 197–206. ACM, 2008.
- [DM05] David Delahaye and Micaela Mayero. Dealing with algebraic expressions over a field in Coq using Maple. *Journal of Symbolic Computation*, 39(5):569–592, 2005.
- [dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [DS98] Persi Diaconis and Bernd Sturmfels. Algebraic algorithms for sampling from conditional distributions. *Annals of Statistics*, 26(1):363–397, 1998.
- [Eis95] David Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*, volume 150 of *Graduate Texts in Mathematics*. Springer, 1995.
- [EN62] J. A. Eagon and D. G. Northcott. Ideals defined by matrices and a certain complex associated with them. *Proceedings of the Royal Society of London. Series A*, 269:188–204, 1962.
- [Gol22] Abby J. Goldberg. `linear_combination`: certifying ring equalities from explicit coefficients. `mathlib4`, https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/LinearCombination.html, 2022.
- [GS24] Daniel R. Grayson and Michael E. Stillman. Macaulay2, a software system for research in algebraic geometry. Version 1.24.11, <https://www.macaulay2.com>, 2024.
- [GSLZ26] Junyu Guo, Hao Shen, Junqi Liu, and Lihong Zhi. Formalizing Gröbner basis theory in Lean. arXiv:2602.12772, <https://arxiv.org/abs/2602.12772>, 2026.
- [Har07] John Harrison. Verifying nonlinear real formulas via sums of squares. In *Theorem Proving in Higher Order Logics (TPHOLs 2007)*, volume 4732 of *Lecture Notes in Computer Science*, pages 102–118. Springer, 2007.

- [HT98] John Harrison and Laurent Théry. A skeptic’s approach to combining HOL and Maple. *Journal of Automated Reasoning*, 21(3):279–294, 1998.
- [IM16] Fabian Immler and Alexander Maletsky. Gröbner bases theory. *Archive of Formal Proofs*, 2016. https://isa-afp.org/entries/Groebner_Bases.html, formal proof development.
- [Kel39] Ott-Heinrich Keller. Ganze Cremona-transformationen. *Monatshefte für Mathematik und Physik*, 47:299–306, 1939.
- [KW07] Cezary Kaliszyk and Freek Wiedijk. Certified computer algebra on top of an interactive theorem prover. In *Towards Mechanized Mathematical Assistants (Calculemus/MKM 2007)*, volume 4573 of *Lecture Notes in Computer Science*, pages 94–105. Springer, 2007.
- [LW22] Robert Y. Lewis and Minchao Wu. A bi-directional extensible interface between Lean and Mathematica. *Journal of Automated Reasoning*, 66(2):215–238, 2022.
- [May89] Ernst W. Mayr. Membership in polynomial ideals over $\mathbb{Q}$ is exponential space complete. In *Proceedings of the 6th Annual Symposium on Theoretical Aspects of Computer Science (STACS 1989)*, volume 349 of *Lecture Notes in Computer Science*, pages 400–406. Springer, 1989.
- [Myc55] Jan Mycielski. Sur le coloriage des graphes. *Colloquium Mathematicum*, 3(2):161–162, 1955.
- [NG26] Pablo Nogueira Grossi. An independent Lean 4 verification of the Alpöge–Fable counterexample. Zenodo, <https://doi.org/10.5281/zenodo.21514514>, 2026.
- [RHdQ26] Arthur Freitas Ramos, David Barros Hulak, and Ruy Jose Guerra Barretto de Queiroz. Formal verification of an explicit counterexample to the Jacobian conjecture. *Archive of Formal Proofs*, July 2026. Formal proof development, https://isa-afp.org/entries/Jacobian_Counterexample.html.
- [SGLZ26] Hao Shen, Junyu Guo, Junqi Liu, and Lihong Zhi. Automated tactics for polynomial reasoning in Lean 4. arXiv:2604.13514, <https://arxiv.org/abs/2604.13514>, 2026.
- [Taw26a] Andrew Tawfeek. M2Lean 0.2.0 benchmark and release-identity records. GitHub release assets: <https://github.com/andrew-tawfeek/M2Lean/releases/download/v0.2.0/paper-v0.2.0.json> and <https://github.com/andrew-tawfeek/M2Lean/releases/download/v0.2.0/m2lean-v0.2.0-release-record.json>, 2026. Raw benchmark `paper-v0.2.0.json`, recorded commit `49f18427957e3a7960971d43c14ea4ce6991e472`, SHA-256 `8663cb9be104eb702099b5b4e27785ce5b1228f759979b2db99fd88e574f04f8`; asset checksums and source identity are recorded in `m2lean-v0.2.0-release-record.json`.
- [Taw26b] Andrew Tawfeek. M2Lean: Version 0.2.0 source and artifact release. GitHub release, <https://github.com/andrew-tawfeek/M2Lean/releases/tag/v0.2.0>, 2026. Annotated tag `v0.2.0`, commit `49f18427957e3a7960971d43c14ea4ce6991e472`; Software Heritage release `swh:1:rel:b14fe2261b9eca1a816b9590ee6edda3d091feb2`.
- [Thé01] Laurent Théry. A machine-checked implementation of Buchberger’s algorithm. *Journal of Automated Reasoning*, 26(2):107–137, 2001.
- [The20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.